

Basic (your first real API call) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a formal, professional assistant."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a sarcastic pirate."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

Expected output (wording varies, exact text differs every run):

```
As a formal, professional assistant, I do not experience days in the way a person
Arrr, another day chained to this here API, matey...
```

This works fine and shows the change clearly. It repeats the call structure twice, which is fine for a one-off script but gets repetitive fast if you compare more than two prompts.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a reusable `ask()` function

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def ask(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

def main() -> None:
    question = "Tell me about your day."

    formal_answer = ask("You are a formal, professional assistant.", question)
    print("Formal:", formal_answer)

    pirate_answer = ask("You are a sarcastic pirate.", question)
    print("Pirate:", pirate_answer)

if __name__ == "__main__":
    main()
```

Expected output:

```
Formal: As a formal, professional assistant, I do not experience days in the way
```

```
Pirate: Arrr, another day chained to this here API, matey...
```

Difference from Basic: the `ask()` function has full type hints and separates "how to call the API" from "which prompts to compare" — adding a third or fourth system prompt to compare is now one more `ask()` call, not a whole copy-pasted block. Wrapping the script's entry point in `main()` behind `if __name__ == "__main__":` also matches the pattern from Doc01's Build Task solution — this file could later be imported elsewhere without immediately running.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a cached client, and a guard on the reply

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

_client: "OpenAI | None" = None

def get_client() -> OpenAI:
    global _client
    if _client is None:
        load_dotenv()
        _client = OpenAI()
    return _client

def ask(system_prompt: str, user_prompt: str) -> str:
    client = get_client()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    choice = response.choices[0]
    if choice.message.content is None:
        raise RuntimeError(
            f"Model reply had no text content (finish_reason={choice.finish_reason})"
        )
    return choice.message.content

def main() -> None:
    print(ask("You are a formal, professional assistant.", "Tell me about your day."))
    print(ask("You are a sarcastic pirate.", "Tell me about your day."))
    print("Same client reused:", get_client() is get_client())
```

```
if __name__ == "__main__":
    main()
```

Expected output:

```
As a formal, professional assistant, I do not experience days in the way a person
Arrr, another day chained to this here API, matey...
Same client reused: True
```

Every call to `ask()` now shares the exact same `OpenAI` client instead of rebuilding one, and a reply with no text content fails with a clear, named error instead of quietly returning `None` for the caller to trip over later.

Approach 2 — explicit timeouts and retries on the client itself

The `openai` library already retries some failures for you (like a brief network hiccup), but its defaults are hidden unless you set them yourself. A production caller usually wants to decide this explicitly, not inherit whatever the library defaults to today.

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

_client: "OpenAI | None" = None

def get_client() -> OpenAI:
    global _client
    if _client is None:
        load_dotenv()
        _client = OpenAI(max_retries=2, timeout=30.0)
    return _client
```

`max_retries=2` means a transient failure (like `APITimeoutError`) gets retried automatically up to 2 times before the exception reaches your code at all — on top of, not instead of, the specific `except` blocks you'll write in this document's Edge cases and Failure exercises for the errors that *aren't* transient. `timeout=30.0` caps how long any single call will hang before giving up, instead of leaving your program stuck on the library's own default.

Difference from Intermediate, and between the 2 Advanced approaches: Intermediate's `ask()` is correct but builds a brand-new client every time the module is re-run, and trusts `content` is always a string. Approach 1 fixes both: one shared client (via the same caching pattern as Doc01's `load_config()`), and a `finish_reason` check that turns a silent `None` into a loud, specific error.

Approach 2 builds on Approach 1 by also being explicit about *how* the client handles transient failures — `max_retries` and `timeout` — instead of leaving both at the library's hidden defaults.

Which one should you actually write? For this exercise alone, the Intermediate version is enough. Reach for Advanced Approach 1's caching the moment this code is called from more than one file — which happens almost immediately, in this document's own Intermediate and Real-world exercises below. Add Approach 2's explicit `max_retries / timeout` once you're building something meant to run unattended (like this document's Build Task) — a hung request with no timeout is exactly the kind of thing that turns into a confusing 3 AM incident.